操作系统实验报告

实验零：环境搭建与实验准备

姓　　名：　郭盈盈

学　　号：　24312063

教学班号：　吴岸聪老师班

专　　业：　保密管理

院　　系：　计算机学院

2025 学年第二学期

一. 实验目的

1. Rust 学习和巩固，了解标准库提供的基本数据结构和功能。
2. QEMU 与 Rust 环境搭建，尝试使用 QEMU 启动 UEFI Shell。
3. 了解 x86 汇编、计算机的启动过程，UEFI 的启动过程，实现 UEFI 下的 Hello, world!。

二. 实验内容

1. 配置实验环境

2. 尝试使用 Rust 进行编程：五个基础任务

2.1. 使用 Rust 编写一个程序,完成以下任务:

a.创建一个函数 count_down(seconds: u64)

该函数接收一个 u64 类型的参数，表示倒计时的秒数。

函数应该每秒输出剩余的秒数，直到倒计时结束，然后输出 Countdown finished!。

b.创建一个函数 read_and_print(file_path: &str)

该函数接收一个字符串参数，表示文件的路径。

函数应该尝试读取并输出文件的内容。如果文件不存在，函数应该使用 expect 方法主动 panic,并输出 File not found!。

c.创建一个函数 `file_size(file_path: &str) -> Result<u64, &str>`

该函数接收一个字符串参数，表示文件的路径，并返回一个 Result。

函数应该尝试打开文件，并在 Result 中返回文件大小。如果文件不存在，函数应该返回一个包含 File not found! 字符串的 Err。

d.在 main 函数中，按照如下顺序调用上述函数:

首先调用 count_down(5) 函数进行倒计时

然后调用 read_and_print("/etc/hosts") 函数尝试读取并输出文件内容

最后使用 std::io 获取几个用户输入的路径，并调用 file_size 函数尝试获取文件大小，并处理可能的错误。

2.2. 实现一个进行字节数转换的函数，并格式化输出:

a.实现函数 humanized_size(size: u64) -> (f64, &‘static str) 将字节数转换为人类可读的大小和单位

使用 1024 进制，并使用二进制前缀（B, KiB, MiB, GiB）作为单位

b.补全格式化代码，使得你的实现能够通过如下测试：

```
#[test]
fn test_humanized_size() {
    let byte_size = 1554056;
    let (size, unit) = humanized_size(byte_size);
    assert_eq!("Size :  1.4821 MiB", format!(/* FIXME */));
}
```

2.3. 自行搜索学习如何利用现有的 crate 在终端中输出彩色的文字

输出一些带有颜色的字符串，并尝试直接使用 print! 宏输出一到两个相同的效果。

尝试输出如下格式和内容： - INFO: Hello, world!，其中 INFO: 为绿色，后续内容为白色 - WARNING: I’m a teapot!，颜色为黄色，加粗，并为 WARNING 添加下划线 - ERROR: KERNEL PANIC!!!，颜色为红色，加粗，并尝试让这一行在控制行窗口居中 - 一些你想尝试的其他效果和内容……

2.4. 使用 enum 对类型实现同一化

实现一个名为 Shape 的枚举，并为它实现 pub fn area(&self) -> f64 方法，用于计算不同形状的面积。 - 你可能需要使用模式匹配来达到相应的功能 - 请实现 Rectangle 和 Circle 两种 Shape，并使得 area 函数能够正确计算它们的面积 - 使得你的实现能够通过如下测试：

```
#[test]
fn test_area() {
    let rectangle = Shape::Rectangle {
        width: 10.0,
        height: 20.0,
    };
    let circle = Shape::Circle { radius: 10.0 };

    assert_eq!(rectangle.area(), 200.0);
    assert_eq!(circle.area(), 314.1592653589793);
}

!!! note "可以使用标准库提供的 `std::f64::consts::PI`"
```

2.5. 实现一个元组结构体 UniqueId(u16)

使得每次调用 UniqueId::new() 时总会得到一个新的不重复的 UniqueId。 - 你可以在函数体中定义 static 变量来存储一些全局状态 - 你可以尝试使用 std::sync::atomic::AtomicU16 来确保多

线程下的正确性（无需进行验证，相关原理将在 Lab 5 介绍，此处不做要求）- 使得你的实现能够通过如下测试：

```
#[test]
fn test_unique_id() {
    let id1 = UniqueId::new();
    let id2 = UniqueId::new();
    assert_ne!(id1, id2);
}
```

3. 运行 UEFI Shell：初始化仓库，使用 QEMU 启动 UEFI Shell

4. YSOS 启动：配置 Rust ToolChain，运行第一个 UEFI 程序

三. 实验过程

1. 配置实验环境

1.1. 安装项目开发环境

1.安装 WSL2

```
wsl --install -d Ubentu
```

2.使用系统包管理器安装依赖

```
je-suis-un-chat@LAPTOP-MAGCR3QA:~$ qemu-system-x86_64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.12)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project develop
ers
je-suis-un-chat@LAPTOP-MAGCR3QA:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There
 is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR P
URPOSE.

je-suis-un-chat@LAPTOP-MAGCR3QA:~$ gdb --version
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/
gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

图 1　验证相关软件包的版本

3.安装 Rust 开发环境与工具链

1.安装 Rustup

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2.安装 Rust Toolchain

```
rustup toolchain install stable-x86_64-unknown-uefi
```

3.安装 Rust 组件

```
rustup component add rust-src --toolchain stable-x86_64-unknown-uefi
rustup component add llvm-tools-preview --toolchain stable-x86_64-unknown-uefi
```

4.检查编译器版本

```
rustc --version
cargo --version
```

```
je-suis-un-chat@LAPTOP-MA

je-suis-un-chat@LAPTOP-MAGCR3QA:~$ rustc --version
rustc 1.93.1 (01f6ddf75 2026-02-11)
je-suis-un-chat@LAPTOP-MAGCR3QA:~$ rustc +nightly --version
rustc 1.96.0-nightly (ec818fda3 2026-03-02)
je-suis-un-chat@LAPTOP-MAGCR3QA:~$
```

2. 尝试使用 Rust 进行编程

这部分要完成五份代码，我通过学习文档和与大模型对话掌握了 rust 的基本语法并完成了这部分任务。

通过任务一，我学会了用 cargo 构建一个 rust 项目，并熟悉了一个标准 rust 项目的代码文件和配置文件以及它们之间的关系。

通过任务二，我了解了 cargo 提供的良好的单元测试、集成测试，掌握了和 C、C++不太一样的调试流程。

通过任务三，我掌握了如何在终端中输出彩色的文字，在完成这部分任务的时候，我还遇到了一个问题：在配置 colored 依赖包并运行 cargo run 时，我一直遭遇网络报错：SSL connect error (unexpected eof while reading)。无论重试多少次，Cargo 都无法成功下载依赖的索引文件。出现这个问题主要是因为在 WSL 环境下，网络拦截的情况比较复杂。为了排查与解决问题，我尝试了几个常见的修复手段：切换清华大学镜像源、更改拉取方式、关闭多路复用，都尝试失败。最终我运行了以下命令来检查网络：

```
curl -v https://index.crates.io/config.json
```

结果显示：

```
    e-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/colored_output$ curl -v https://
index.crates.io/config.json
* Uses proxy env variable no_proxy ==
'192.168.*,172.31.*,172.30.*,172.29.*,172.28.*,172.27.*,172.26.*,172.25.*,172.24.*,
172.23.*,172.22.*,172.21.*,172.20.*,172.19.*,172.18.*,172.17.*,172.16.*,10.*,127.*,
localhost,<local>'
* Uses proxy env variable https_proxy == 'https://127.0.0.1:7890'
*   Trying 127.0.0.1:7890...
* Connected to 127.0.0.1 (127.0.0.1) port 7890
* ALPN: curl offers http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
*  CAfile: /etc/ssl/certs/ca-certificates.crt
*  CApath: /etc/ssl/certs
* OpenSSL SSL_connect: SSL_ERROR_SYSCALL in connection to 127.0.0.1:7890
* Closing connection
curl: (35) OpenSSL SSL_connect: SSL_ERROR_SYSCALL in connection to 127.0.0.1:7890
je-suis-un-chat@LAPTOP-MAGCR3QA:~/YatSenOS/colored_output$
```

意味着根本问题出在我的终端代理环境变量格式写错了。

最终，我用用 unset https_proxy、unset http_proxy 等命令，清除了当前环境中错误的代理变量。并重新以正确的格式声明了代理：将 https_proxy 和 http_proxy 的值都设置为了 http://127.0.0.1:7890。

修改完成之后，网络瞬间打通，我的彩色输出也顺利跑起来了。

通过任务四，我学会了 rust 语句中的两个重要概念：携带数据的枚举和模式匹配。在 rust 中，枚举的每一个变体都可以独立携带属于自己的数据结构，而 match 可以在需要时安全地把结构解构并分类处理。

通过任务五，我了解了 rust 的元组结构体—-只有一个类型包裹着的结构体，和全局状态量 static—-储存在程序固定的内存区域，贯穿程序整个生命周期。而 AtomicU16 可以保证每一次获取的值都各不相同。

3. 运行 UEFI Shell

3.1. 概念学习

由于这部分内容涉及到很多不认识没见过的名词，所以在任务开始前，我先通过网络搜索和于大模型对话的方式，了解了一些基本概念，笔记如下:

3.1.1. UEFI

UEFI，即统一可扩展固件接口，是操作系统和固件之间的接口，它提供了启动操作系统或运行预启动程序的标准环境。它支持多种操作系统的加载，如 Windows、Linux 等。在实验文档的上下文中，UEFI Shell 是一个基于 UEFI 的命令行工具，它允许我们在 UEFI 环境中执行一些简单的操作，例如查看硬件信息、启动操作系统等。

3.1.2. QEMU

QEMU 是一个开源的托管虚拟机软件，它可以模拟各种硬件平台和设备，使得一台电脑能模拟运行另一台完全不同的电脑系统，它主要有以下两大核心功能:

1.全系统模拟:

在这个模式下，QEMU 会模拟一整套硬件，包括 CPU、内存、硬盘和网卡。这使得用户可以在一个物理机上运行另一个操作系统，就像在真实硬件上一样。

2.用户模式模拟:

在这个模式下，QEMU 只模拟用户空间的程序，而不模拟整个系统。这使得用户可以在一个平台上运行另一个平台的应用程序，例如在 x86 上运行 ARM 程序。

3.1.3. OVMF

OVMF 是一个开源的 UEFI 固件实现，它允许我们在 QEMU 中模拟 UEFI 环境。通过使用 OVMF，我们可以在 QEMU 中启动 UEFI Shell，并进行一些基本的操作，例如查看硬件信息、启动操作系统等。

3.2. 初始化你的仓库

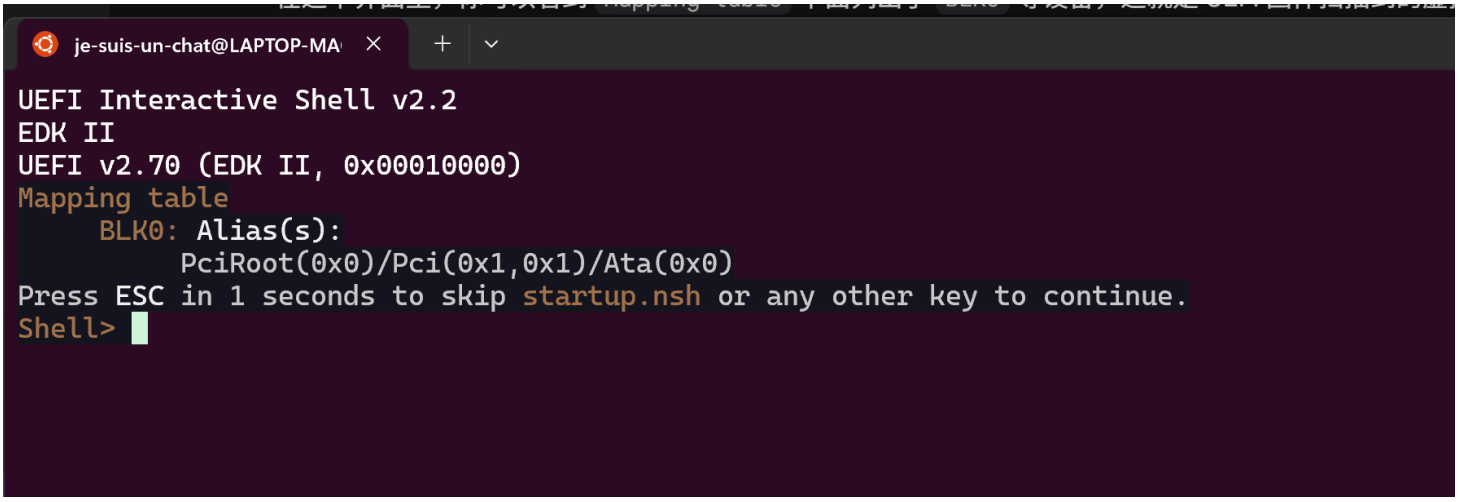
首先将实验仓库克隆到本地，然后参考实验 0x00 代码的文件结构，初始化我的仓库。

3.3. 使用 QEMU 启动 UEFI Shell

UEFI Shell 是一个基于 UEFI 的命令行工具，它可以让我们在 UEFI 环境下进行一些简单的操作。

在不挂载任何硬盘的情况下，我们可以使用如下命令启动 UEFI Shell：

```
qemu-system-x86_64 -bios ./assets/OVMF.fd -net none -nographic
```



命令参数解释：

`qemu-system-x86_64`：指定使用 QEMU 模拟 x86_64 架构的系统。

`-bios ./assets/OVMF.fd`：指定使用 OVMF 固件文件作为 BIOS，这个文件提供了 UEFI 环境。

`-net none`：禁用网络功能。

`-nographic`：禁用图形输出，使用命令行界面。

4. YSOS 启动！

4.1. 配置 Rust ToolChain

由于仓库提供的 rust-toolchain.toml 已经指定了需要使用的 Rust 工具链版本，只需要将这个文件放在项目根目录下，运行 cargo build 就会自动下载并使用指定的工具链。

4.2. 运行第一个 UEFI 程序

在 crates/boot/src/main.rs 中，完善示例代码，修改注释部分，完整代码如下：

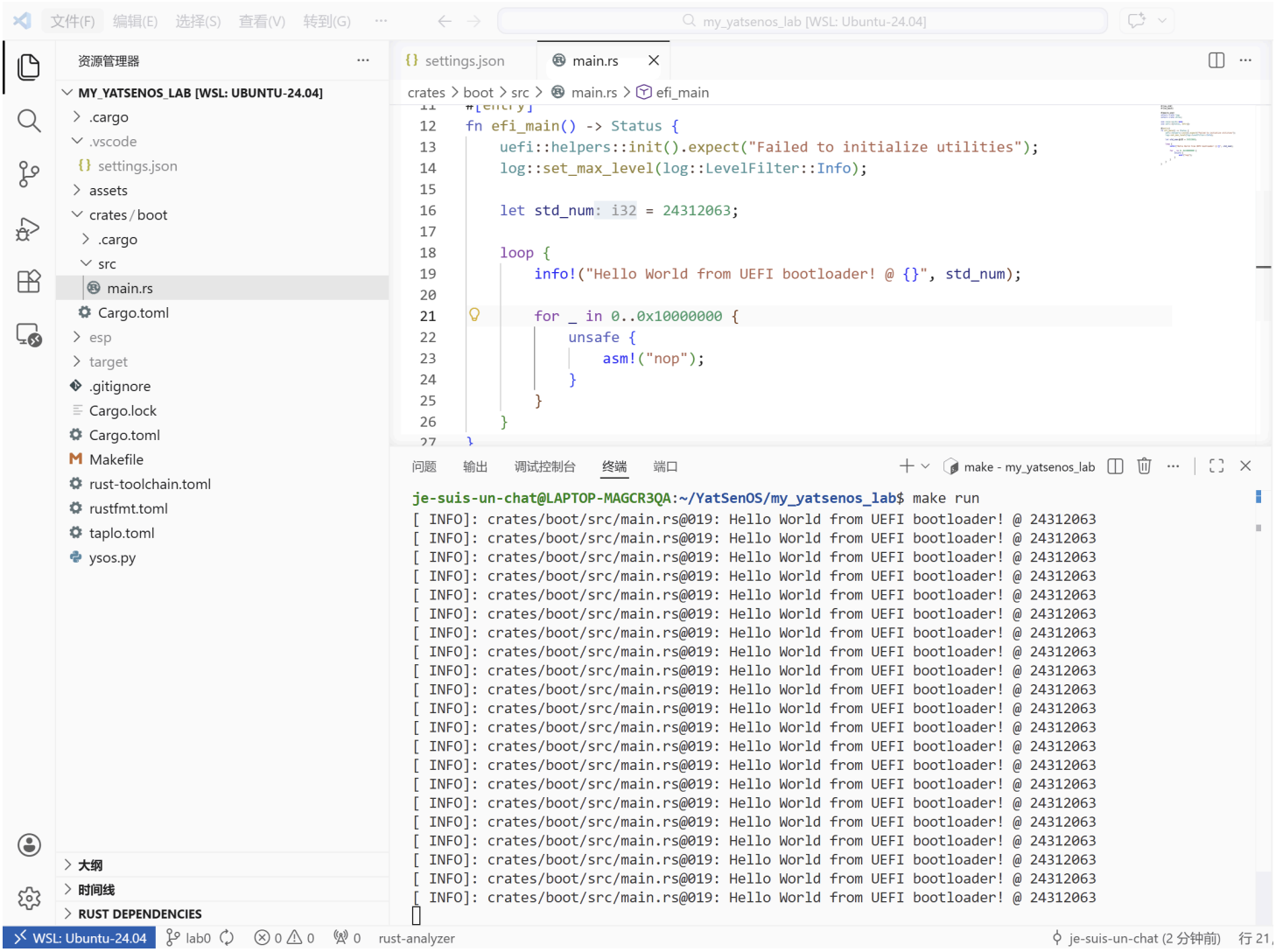
```rust
#![no_std]
#![no_main]

#[macro_use]
extern crate log;
extern crate alloc;

use core::arch::asm;
use uefi::{Status, entry};

#[entry]
fn efi_main() -> Status {
    uefi::helpers::init().expect("Failed to initialize utilities");
    log::set_max_level(log::LevelFilter::Info);

    let std_num = 24312063;

    loop {
        info!("Hello World from UEFI bootloader! @ {}", std_num);

        for _ in 0..0x10000000 {
            unsafe {
                asm!("nop");
            }
        }
    }
}
```

由于本人初学 rust，所以需要对这份简单代码进行分析，笔记如下：

1. `#![no_std]` 和 `#![no_main]` 是 Rust 的属性宏，分别表示不使用标准库和不使用默认的 main 函数作为程序入口点。这在操作系统开发中很常见，因为我们需要直接控制底层硬件，而不依赖于标准库提供的功能。
2. `#[macro_use] extern crate log;` 和 `crate alloc;` 是引入外部 crate 的语句，分别引入了日志库和分配器库。这些库提供了日志记录和内存分配的功能，在操作系统开发中非常有用。
3. `core::arch::asm;` 引入了 Rust 的内联汇编功能，允许我们在 Rust 代码中直接编写汇编指令。
4. `uefi::{Status, entry};` 引入了 UEFI crate 中的 Status 类型和 entry 宏，Status 用于表示函数的返回状态，而 entry 宏用于定义程序的入口点。
5. `#[entry] fn efi_main() -> Status { ... }` 定义了程序的入口函数 efi_main，这个函数会在 UEFI 环境下被调用。函数返回一个 Status 类型的值，表示函数执行的结果。
6. 在 efi_main 函数中，首先调用了 `::helpers::init()` 来初始化 UEFI 的实用工具，如果初始化失败会 panic 并输出错误信息。 接着设置了日志级别为 Info。
7. 定义了一个变量 std_num，赋值为我的学号 24312063。

8. 进入一个无限循环，在循环中使用 `!` 宏输出一条日志消息，包含 “Hello World from UEFI bootloader!” 和我的学号。然后通过一个大循环来创建一个延迟，使用内联汇编的 nop 指令来占用 CPU 时间，防止程序过快地输出日志消息。

得到如下期望输出：



至此，我已经做好了编写 OS 的准备工作。

四. 思考题

1. 了解现代操作系统（Windows）的启动过程，UEFI 和 Legacy（BIOS）的区别是什么？

答:UEFI（统一可扩展固件接口）和 Legacy BIOS（基本输入输出系统）是两种不同的计算机启动固件接口。它们之间的主要区别如下：

==- 启动方式：Legacy BIOS 使用 MBR（主引导记录）分区方案，而 UEFI 使用 GPT（GUID 分区表）分区方案。UEFI 支持更大的硬盘和更多的分区。

- 启动速度：UEFI 启动速度通常比 Legacy BIOS 快，因为它使用更现代的启动机制和更高效的代码。
- 安全性：UEFI 支持 Secure Boot（安全启动），可以防止未经授权的操作系统和驱动程序加载，提高系统安全性。Legacy BIOS 没有这种功能。
- 用户界面：UEFI 通常提供更友好的图形用户界面，而 Legacy BIOS 主要是基于文本的界面。

2. 尝试解释 Makefile 中的命令做了哪些事情？或许你可以参考下列命令来得到更易读的解释：

```
python ysos.py run --dry-run
```

答:这个命令是使用 Python 运行一个名为 ysos.py 的脚本，并传递了两个参数：run 和 –dry-run。

- run 参数可能是告诉脚本执行某个特定的操作，可能是运行一个程序或者执行某个任务。
- –dry-run 参数通常用于模拟执行命令而不实际执行它。这意味着脚本会显示它将要执行的操作，但不会真正执行任何更改。这对于测试和验证命令的正确性非常有用，可以帮助用户了解命令的效果而不冒风险。

3. 利用 cargo 的包管理和 docs.rs 的文档，我们可以很方便的使用第三方库。这些库的源代码在哪里？它们是什么时候被编译的？

答:第三方库的源代码通常托管在 GitHub 或其他代码托管平台上。当我们使用 cargo 来添加依赖时，cargo 会从 crates.io（Rust 的包注册中心）下载库的源代码，并将其存储在本地的 cargo 缓存目录中（通常位于 /.cargo/registry/src/）。这些库是在我们第一次构建项目时被编译的，或者当我们更新依赖时重新编译。编译后的库文件会被存储在 target 目录下的 debug 或 release 文件夹中，具体取决于我们使用的是 debug 模式还是 release 模式。

4. 为什么我们需要使用 entry 而不是直接使用 main 函数作为程序的入口？

答:在 Rust 中，entry 是一个属性宏，用于指定程序的入口点。我们需要使用 entry 而不是直接使用 main 函数作为程序的入口，是因为在某些环境（如嵌入式系统或操作系统开发）中，main 函数可能不适合作为入口点。entry 允许我们定义一个自定义的入口函数，这个函数可以根据特

定的环境需求进行配置和初始化，而不受 main 函数的限制。这对于操作系统开发来说尤其重要，因为我们可能需要在程序启动时执行一些特定的初始化步骤，而这些步骤可能不适合放在 main 函数中。